Лабораторная работа №9

Сетевое взаимодействие в Python: TCP/UDP, клиент-серверная архитектура и прикладные протоколы

1. Цель работы

Изучить взаимодействие TCP-клиента и сервера, JSON-протокол, многопоточную обработку соединений и тестирование прикладного сетевого сервиса такси.

2. Постановка индивидуального задания

Вариант 12 — «Такси». Разработать сетевой сервис управления автомобилями такси.

Программа должна:

– Разработать TCP-сервер и консольный TCP-клиент для учёта автомобилей такси.

– Реализовать JSON-команды list, get, create, update, delete, free.

– Обеспечить работу с несколькими клиентами, обработку ошибок и автоматические тесты.

3. Краткие теоретические сведения

При клиент-серверной архитектуре клиент отправляет запросы, а сервер выполняет операции с данными и возвращает ответы. IP-адрес задаёт узел сети, порт идентифицирует приложение. В данном проекте при ручном запуске используется 127.0.0.1:5012.

TCP обеспечивает упорядоченный поток байтов, но не сохраняет границы сообщений. Программа отделяет JSON-документы переводом строки; функция encode() формирует сообщение, decode() восстанавливает словарь. UDP ориентирован на независимые датаграммы и в основной реализации не применяется.

Server на основе ThreadingTCPServer обрабатывает несколько клиентов. Доступ к общему хранилищу синхронизирован RLock. Типовой ответ JSON

содержит status и data либо status и error. В обработчиках используются ограничения времени ожидания.

4. Исходный код программы

Файл domain.py

```
from dataclasses import dataclass, asdict
from threading import RLock


class TaxiError(ValueError):
    pass


@dataclass
class Car:
    id: int
    number: str
    driver: str
    seats: int
    available: bool = True


class CarRepository:
    def __init__(self) -> None:
        self.items: dict[int, Car] = {}

    def get(self, id: int) -> Car:
        if id not in self.items:
            raise TaxiError("Автомобиль не найден")
        return self.items[id]

    def get_all(self) -> list[Car]:
        return list(self.items.values())

    def add(self, car: Car) -> None:
        if car.id in self.items:
            raise TaxiError("id уже существует")
        self.items[car.id] = car

    def delete(self, id: int) -> None:
        self.get(id)
        del self.items[id]


class TaxiService:
    def __init__(self, repository: CarRepository) -> None:
        self.repository = repository
        self.lock = RLock()

    def validate(self, car: Car) -> None:
        if type(car.id) is not int or car.id <= 0:
            raise TaxiError("id должен быть положительным целым")
        if type(car.seats) is not int or car.seats <= 0:
            raise TaxiError("Количество мест должно быть положительным целым")
        if not isinstance(car.number, str) or not car.number.strip():
            raise TaxiError("Укажите номер автомобиля")
        if not isinstance(car.driver, str) or not car.driver.strip():
            raise TaxiError("Укажите водителя")
```

```
        if type(car.available) is not bool:
            raise TaxiError("available должен быть true или false")

    def execute(self, command: str, data: dict) -> object:
        with self.lock:
            if command == "list":
                return [asdict(car) for car in self.repository.get_all()]
            if command == "free":
                return [asdict(car) for car in self.repository.get_all() if car.available]
            if command == "create":
                car = Car(**data)
                self.validate(car)
                self.repository.add(car)
                return asdict(car)
            if command == "get":
                return asdict(self.repository.get(data["id"]))
            if command == "update":
                self.repository.get(data["id"])
                car = Car(**data)
                self.validate(car)
                self.repository.items[car.id] = car
                return asdict(car)
            if command == "delete":
                self.repository.delete(data["id"])
                return None
            raise TaxiError("Неизвестная команда")
```

Файл protocol.py

```
import json
from domain import TaxiError


class ProtocolError(ValueError):
    pass


def encode(message: dict) -> bytes:
    # Перевод строки отделяет одно JSON-сообщение от другого.
    return (json.dumps(message, ensure_ascii=False) + "\n").encode("utf-8")


def decode(line: bytes) -> dict:
    try:
        message = json.loads(line)
    except (ValueError, UnicodeError) as error:
        raise ProtocolError("Некорректный JSON") from error
    if not isinstance(message, dict):
        raise ProtocolError("Ожидался JSON-объект")
    return message


def handle(line: bytes, service) -> dict:
    try:
        request = decode(line)
        command, data = request["command"], request["data"]
```

```
        if not isinstance(command, str) or not isinstance(data, dict):
            raise ProtocolError("command — строка, data — объект")
        return {"status": "ok", "data": service.execute(command, data)}
    except (ProtocolError, TaxiError, KeyError, TypeError) as error:
        return {"status": "error", "error": str(error)}
```

Файл server.py

```
import logging
import socketserver
from domain import CarRepository, TaxiService
from protocol import encode, handle

service = TaxiService(CarRepository())


class Handler(socketserver.StreamRequestHandler):
    def handle(self) -> None:
        logging.info("Подключён клиент %s", self.client_address)
        self.request.settimeout(30)
        try:
            for line in self.rfile:
                response = handle(line, service)
                logging.info("Запрос %s; ответ %s", line.decode(errors="replace").strip(), response)
                self.wfile.write(encode(response))
                self.wfile.flush()
        except (OSError, ValueError):
            logging.exception("Соединение завершено или истёк тайм-аут")
        finally:
            logging.info("Клиент отключён %s", self.client_address)


class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True


if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")
    with Server(("127.0.0.1", 5012), Handler) as server:
        logging.info("TCP-сервер: 127.0.0.1:5012")
        try:
            server.serve_forever()
        except KeyboardInterrupt:
            pass
```

Файл client.py

```
import socket
from protocol import encode, decode


def main() -> None:
    try:
        with socket.create_connection(("127.0.0.1", 5012), timeout=5) as connection:
            with connection.makefile("rb") as reader:
```

```
            while True:
                print("\nlist — все; get — один; create — создать; update — изменить")
                print("delete — удалить; free — свободные автомобили; 0 — выход")
                command = input("Команда: ")
                if command == "0":
                    break
                try:
                    data = {}
                    if command in ("get", "delete", "create", "update"):
                        data["id"] = int(input("id: "))
                    if command in ("create", "update"):
                        data.update(number=input("Номер: "), driver=input("Водитель: "),
                                    seats=int(input("Места: ")), available=input("Свободен? 1/0: ") == "1")
                    connection.sendall(encode({"command": command, "data": data}))
                    line = reader.readline()
                    if not line:
                        print("Сервер закрыл соединение")
                        break
                    print(decode(line))
                except ValueError as error:
                    print("Ошибка ввода:", error)
    except OSError as error:
        print("Ошибка соединения или тайм-аут:", error)


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 9.1 приведён фрагмент контрольного запуска обмена JSON-сообщениями между TCP-клиентом и сервером.

```
python client.py
Сервер TCP: 127.0.0.1:5012
Запрос: {"command": "create", "data": {"id": 1, "number": "A001AA", "driver":
"Иван", "seats": 4, "available": true}}
Ответ:  {"status": "ok", "data": {"id": 1, "number": "A001AA", "driver":
"Иван", "seats": 4, "available": true}}
Запрос: {"command": "create", "data": {"id": 2, "number": "A002AA", "driver":
"Анна", "seats": 4, "available": false}}
Ответ:  {"status": "ok", "data": {"id": 2, "number": "A002AA", "driver":
"Анна", "seats": 4, "available": false}}
Запрос: {"command": "free", "data": {}}
Ответ:  {"status": "ok", "data": [{"id": 1, "number": "A001AA", "driver":
"Иван", "seats": 4, "available": true}]}
Запрос: {"command": "get", "data": {"id": 1}}
Ответ:  {"status": "ok", "data": {"id": 1, "number": "A001AA", "driver":
"Иван", "seats": 4, "available": true}}
Запрос: {"command": "delete", "data": {"id": 2}}
Ответ:  {"status": "ok", "data": null}
Запрос: {"command": "list", "data": {}}
Ответ:  {"status": "ok", "data": [{"id": 1, "number": "A001AA", "driver":
"Иван", "seats": 4, "available": true}]}
```

Рисунок 9.1 – Результат контрольного запуска программы

Автоматическое тестирование выполнено с помощью pytest. Все 7 тестов прошли успешно (рисунок 9.2).

```
pytest -q
.......
[100%]
7 passed in 0.59s
```

Рисунок 9.2 – Результат автоматического тестирования

6. Вывод

Создан TCP-сервис учёта автомобилей такси с JSON-протоколом, многопоточным сервером, проверкой данных и обработкой ошибок. Подтверждены CRUD-команды, фильтрация доступных машин и прохождение 7 тестов.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	